Emergent Universe Engine

A Comprehensive Pedagogical Review

From quantum-inspired fields to evolving neural organisms:
a ground-up walkthrough of every layer of the simulation

Covering: reaction-diffusion systems, orthogonal-gate physics, thermodynamics on grids, connected-component biology,
model-based neural agents, evolutionary algorithms, social networks, and in-universe symbolic regression.

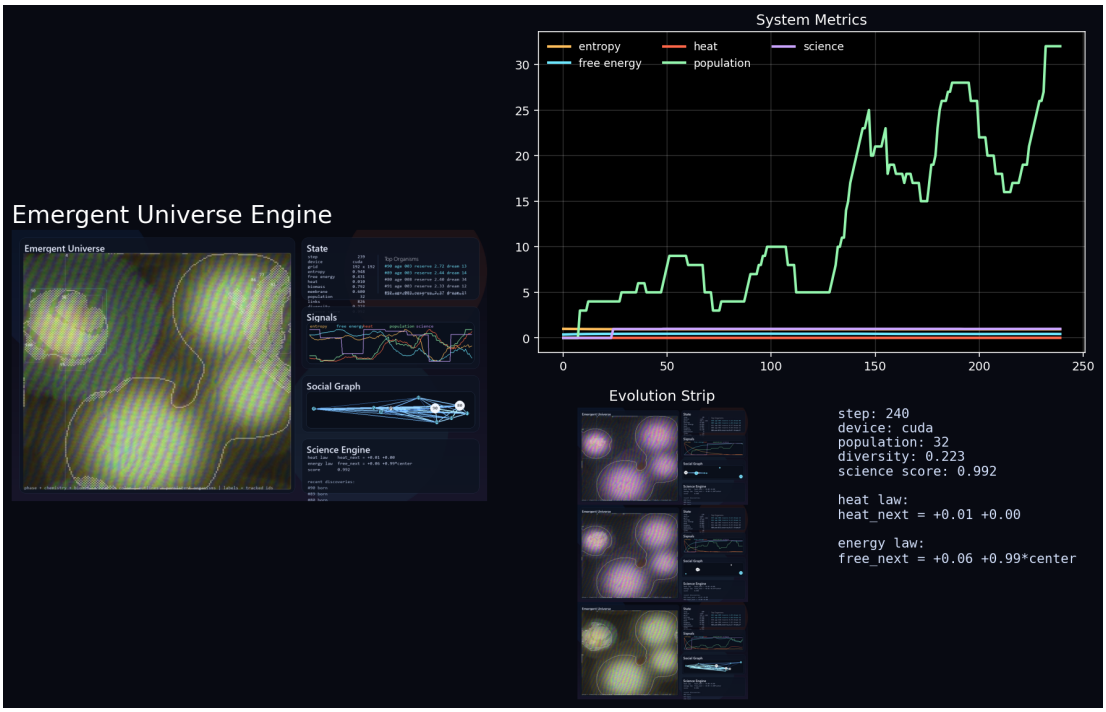

Figure 1: Final-frame poster from a 240-step GPU run of the Emergent Universe Engine (seed 7, 192 × 192 grid, CUDA). The large panel shows the live dashboard; top-right shows evolving system metrics; bottom-right records the discovered physical laws.

Compiled from source: `toy_universe.py`, `physics_engine.py`, `thermodynamics.py`, `cellular_life.py`, `organism.py`, `neural_brain.py`, `evolution.py`, `social_network.py`, `science_engine.py`, `visualization.py`.

Contents

1 What Is Emergence, and Why Simulate a Universe? 3
1.1 A Bird's-Eye View of the Architecture . . . 3

2 The State: What Does the Grid Remember? 3
2.1 Coordinate System . . . 3
2.2 The Fields . . . 4

3 The Physics Engine: A Quantum-Inspired Substrate 4
3.1 Why Does the Simulation Need a "Physics" Layer? . . . 4
3.2 What Is the Psi Field? . . . 4
3.3 Orthogonal Gate Evolution . . . 5
3.4 Diffusion, Mixing, Curl, and Drive . . . 6
3.5 What Does the Psi Field Produce? . . . 7

4 Thermodynamics: Energy, Heat, and the Drive to Live 7
4.1 Why Thermodynamics? . . . 7
4.2 Source Map: Where Does Energy Come From? . . . 8
4.3 Edge Cooling . . . 8
4.4 The Thermodynamic Update Equations . . . 8

5 The Chemical Life Engine: Gray-Scott Reaction-Diffusion 9
5.1 A Brief History of Patterns from Nothing . . . 9
5.2 The Gray-Scott Model . . . 9
5.3 The Extended Gray-Scott in This Simulation . . . 10
5.4 Biomass and Membrane . . . 11
5.5 Signal Field . . . 11

6 Organism Manager: Detecting Life from the Field 11
6.1 The Life Mask . . . 11
6.2 Connected-Component Labelling . . . 12
6.3 Organism Record Lifecycle . . . 12

7 Genomes: The Heritable Blueprint 13
7.1 Scalar Traits . . . 13
7.2 Neural Weight Traits . . . 13
7.3 Initialisation and Mutation . . . 13

8 The Neural Brain: Perception, Action, and Dreaming 14
8.1 Observation: What Can an Organism See? . . . 14
8.2 Action: The Neural Forward Pass . . . 14
8.3 Reward and Memory . . . 16
8.4 The World Model: Learning to Predict . . . 16
8.5 Dreaming: Mental Simulation and Bias Learning . . . 16

9 The Evolution Engine: Darwin on a Grid 17
9.1 Reproduction Conditions . . . 17
9.2 Partner Selection and Genome Blending . . . 17

9.3 Child Placement and Field Seeding . . . 17
9.4 Death . . . 18
9.5 What Does Evolution Select For? . . . 18

10 Social Networks: Trust, Cooperation, and Competition 18
10.1 Building the Social Graph . . . 18
10.2 Cooperation and Competition . . . 19
10.3 Knowledge Transfer . . . 19

11 The Science Engine: In-Universe Symbolic Regression 19
11.1 The Idea: Organisms That Do Physics . . . 19
11.2 Data Collection . . . 19
11.3 Evolutionary Candidate Law Search . . . 20
11.4 Rendering Laws as Text . . . 20

12 Visualisation: Painting the Universe 21
12.1 The Rendering Pipeline . . . 21
12.2 Dashboard Layout . . . 21

13 Reading and Analysing the Simulation Images 22
13.1 The Toy Universe Showcase (Figure 3) . . . 22
13.2 The Full Emergent Universe Engine (Figure 4) . . . 23

14 The Toy Universe vs the Full Engine: A Comparison 25

15 Key Mathematical Concepts — Quick Reference 25
15.1 The Discrete Laplacian and Diffusion . . . 25
15.2 The Sigmoid and Its Relatives . . . 26
15.3 Shannon Entropy . . . 26
15.4 Orthogonal Matrices from QR Decomposition . . . 26
15.5 Gaussian Deposition . . . 26
15.6 Connected-Component Labelling via BFS . . . 27

16 Physical Analogies and Inspirations 27

17 End-to-End Walk-Through of One Simulation Step 27

18 Conclusion: What Has the Simulation Achieved? 28

1 What Is Emergence, and Why Simulate a Universe?

Let us start with a question that sounds almost philosophical: *where do complex things come from?* Crystals form from liquid water. Tornadoes arise from warm air. Brains grow from a single fertilised cell. None of these outcomes were explicitly programmed into their constituents — yet they reliably appear whenever the right conditions are met. This phenomenon is called **emergence**: global patterns that arise from local rules, without any central authority dictating the outcome.

The key intuition about emergence

Emergence is not magic, and it is not mysterious (well, not entirely). It happens whenever:

1. Local interactions between simple elements obey a small set of rules.
2. There is some form of *feedback* — outputs influence future inputs.
3. The system is driven away from equilibrium by a continuous energy source.

Given these three ingredients, self-organisation is almost *inevitable*. The question is not whether structure will appear, but which structure.

The **Emergent Universe Engine** is a simulation that stacks multiple layers of physics, chemistry, biology, cognition, and even a rudimentary in-universe science program on top of one another, each layer feeding into the next. The goal is not to simulate any particular real system faithfully, but to build the richest possible petri dish for emergence to occur, and then watch what happens.

1.1 A Bird's-Eye View of the Architecture

Figure 2 shows the engine's data flow. At the bottom sits a grid of $N \times N$ cells (by default $N = 192$). Every cell carries several real-valued fields that evolve each time-step. Seven engines are responsible for updating these fields, in order:

The output of each step is rendered to screen using an HSV-coloured composite image. Every 4 steps, a frame is captured for the output GIF. The whole simulation runs for 360 steps by default.

2 The State: What Does the Grid Remember?

Before we dive into the engines, let us understand what *state* they are operating on. Think of the 192×192 grid as a fictional universe made of tiny cells, each of which stores a small table of numbers.

2.1 Coordinate System

The grid is parameterised by normalised coordinates $x, y \in [-1, 1]$, with the radius

$$r = \sqrt{x^2 + y^2}$$

computed once at initialisation and stored. This lets the simulation cheaply define radially symmetric structures (e.g., edge cooling that increases toward the boundary, or a source map concentrated near certain "star" positions).

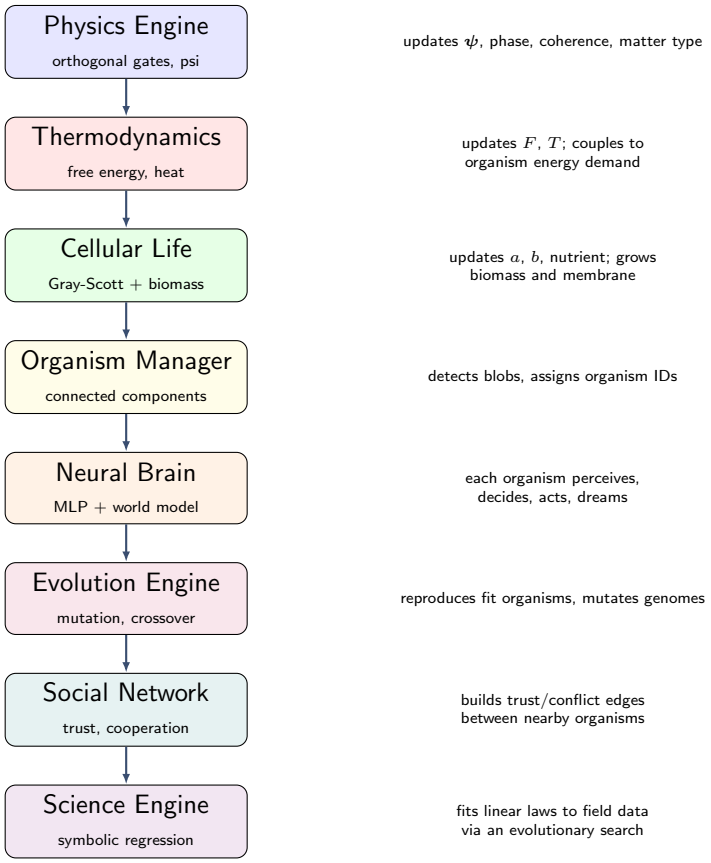

Figure 2: Data-flow diagram of the eight engines that run sequentially each simulation step, from the physics substrate at the top to the in-universe science engine at the bottom.

2.2 The Fields

Additionally three *drive* fields (energy demand, signal drive, repair drive) are written by organisms each step and then cleared: they act as a message-passing layer from the discrete organism layer to the continuous field layer.

3 The Physics Engine: A Quantum-Inspired Substrate

3.1 Why Does the Simulation Need a "Physics" Layer?

In our real universe, all chemistry is built on top of quantum mechanics. We need the simulation's chemistry, life, and cognition to have something similarly "deep" to rest on — a substrate that is rich enough to produce varied patterns without being so simple that everything looks the same.

The physics engine provides this substrate in the form of a four-component complex-like field $\boldsymbol{\psi}$, evolved by rules inspired by (but not identical to) quantum mechanics.

3.2 What Is the Psi Field?

At each grid cell we store a vector

$$\boldsymbol{\psi}(x, y) = (\psi_0, \psi_1, \psi_2, \psi_3) \in \mathbb{R}^4.$$

Field	Shape	Meaning
$\boldsymbol{\psi}$	$4 \times N \times N$	Internal quantum-inspired field
phase φ	$1 \times N \times N$	$\arctan(\psi_1/\psi_0)$
coherence κ	$1 \times N \times N$	tanh-squashed $\lvert\psi_{0:2}\rvert$
matter type m	$1 \times N \times N$	Index of dominant $\lvert\psi_i\rvert$ channel
a, b, nutrient	$3 \times N \times N$	Chemical species (Gray-Scott + nutrient)
free energy F	$1 \times N \times N$	Usable energy density
heat T	$1 \times N \times N$	Thermal energy (waste)
biomass β	$1 \times N \times N$	Life density
membrane μ	$1 \times N \times N$	Cell-boundary density
signal s	$1 \times N \times N$	Chemical communication field
trail	$3 \times N \times N$	Exponential moving average of RGB render

Table 1: Primary fields of the universe state. All fields live on a 192×192 grid and are stored as single-precision floats on GPU (CUDA) or CPU.

Think of this as four independent wave-like amplitudes living at every point. Together they define a local "direction" in an abstract 4-dimensional space. The key physical observables extracted from $\boldsymbol{\psi}$ are:

$$\text{phase:} \quad \varphi = \arctan\left(\frac{\psi_1}{\psi_0}\right), \tag{1}$$

$$\text{coherence:} \quad \kappa = \frac{1}{2} + \frac{1}{2}\tanh\big(1.7\,\|(\psi_0, \psi_1)\| - 0.9\big), \tag{2}$$

$$\text{matter type:} \quad m = \arg\max_i |\psi_i|. \tag{3}$$

Intuition for phase and coherence

Imagine ψ_0 and ψ_1 as the real and imaginary parts of a complex number $z = \psi_0 + i\psi_1$. The *phase* φ is the angle of z in the complex plane — it tells you which direction the field points, and it varies smoothly across space, creating interference-like patterns. The *coherence* κ measures how "concentrated" the field vector is: high coherence ($\kappa \approx 1$) means ψ_0 and ψ_1 together have large magnitude, like a highly ordered crystalline region. Low coherence ($\kappa \approx 0$) means the field is chaotic, like a hot gas. These two quantities — phase and coherence — are then used by the cellular-life engine to modulate where life can grow.

3.3 Orthogonal Gate Evolution

The physics engine applies a brick-wall circuit of *orthogonal linear transformations* to neighbouring columns or rows of cells, alternating every step.

Why orthogonal? An orthogonal matrix $Q \in \mathbb{R}^{d\times d}$ satisfies $Q^T Q = I$, meaning it preserves the Euclidean norm: $\|Q\mathbf{v}\| = \|\mathbf{v}\|$ for all $\mathbf{v}$. This is the real-valued analogue of a unitary transformation in quantum mechanics. Using orthogonal matrices keeps the field from exploding or collapsing to zero — it conserves the local "amplitude budget."

At initialisation two orthogonal matrices are created: $Q_H \in \mathbb{R}^{8\times 8}$ (horizontal gate) and $Q_V \in \mathbb{R}^{8\times 8}$ (vertical gate). They are drawn from the Haar measure by QR-decomposing a random Gaussian matrix and adjusting the sign so the determinant is $+1$.

The gate application. On even steps, adjacent columns j and $j+1$ are paired, their $\boldsymbol{\psi}$ values stacked into a vector of length $2d = 8$, multiplied by Q_H, and the result written back. On odd steps the same is done for adjacent rows with Q_V. The pairing shifts by one cell every two steps ("brick-wall" pattern) to avoid creating frozen edges.

Formally, for a horizontal step pairing columns j and $j+1$:

$$\begin{pmatrix} \boldsymbol{\psi}'(\cdot, j) \\ \boldsymbol{\psi}'(\cdot, j+1) \end{pmatrix} = Q_H \begin{pmatrix} \boldsymbol{\psi}(\cdot, j) \\ \boldsymbol{\psi}(\cdot, j+1) \end{pmatrix}.$$

3.4 Diffusion, Mixing, Curl, and Drive

After the gate step, four additional terms evolve $\boldsymbol{\psi}$:

The full psi update

$$\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} + \underbrace{0.18\,\nabla^2\boldsymbol{\psi}}_{\text{diffusion}} + \underbrace{0.14\,M\boldsymbol{\psi}}_{\text{channel mix}} + \underbrace{0.06\,\mathcal{C}(\boldsymbol{\psi})}_{\text{curl}} + \underbrace{0.11\,\sin(M\boldsymbol{\psi}\cdot 1.8 + \phi_t)}_{\text{oscillatory drive}} - \underbrace{0.12\,\boldsymbol{\psi}}_{\text{damping}} \quad (4)$$

followed by per-cell L^2 normalisation: $\boldsymbol{\psi} \leftarrow \boldsymbol{\psi}/(\|\boldsymbol{\psi}\| + \epsilon)$.

Let us unpack each term:

Diffusion: $0.18\,\nabla^2\boldsymbol{\psi}$**.** The discrete Laplacian

$$(\nabla^2 f)_{i,j} = f_{i-1,j} + f_{i+1,j} + f_{i,j-1} + f_{i,j+1} - 4\,f_{i,j}$$

implemented via a 3×3 convolution kernel $\left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{smallmatrix}\right]$ with circular padding. Diffusion smooths the field and transports information between neighbouring cells — it is the fundamental mechanism behind *all* the reaction-diffusion phenomena we will encounter later.

Channel mix: $0.14\,M\boldsymbol{\psi}$**.** $M \in \mathbb{R}^{4\times 4}$ is a fixed orthogonal matrix ("`mix`" in the code) applied *across the four channels* at each cell independently. This is like a local rotation in the abstract 4D field space: the four amplitudes are continuously blended, creating richer interference patterns than any single channel could alone.

Curl: $0.06\,\mathcal{C}(\boldsymbol{\psi})$**.** The curl term computes

$$\mathcal{C}(\boldsymbol{\psi}) = \mathrm{roll}(\boldsymbol{\psi}, +1, \mathrm{row}) - \mathrm{roll}(\boldsymbol{\psi}, -1, \mathrm{col}),$$

which is a discrete approximation of a curl-like operation. It injects a vortex-like bias into the dynamics, breaking the left-right / up-down symmetry and causing the field to develop swirling, spiral structures.

Oscillatory drive: $0.11\sin(\ldots)$**.** The time-varying phase bias $\phi_t = \sin\big(\phi_b \cdot t \cdot 0.021\big)$ (where $\phi_b = [0, 0.11, 0.22, 0.33, \ldots]$ is a fixed per-channel offset) ensures no two channels oscillate in phase. The nonlinear $\sin(\cdot)$ means the field is kicked in a bounded way — it can never grow unboundedly from this term.

Damping: $-0.12\,\boldsymbol{\psi}$**.** Without damping, the accumulation of all those terms would let $\boldsymbol{\psi}$ grow without bound (before normalisation). The damping term $-0.12\boldsymbol{\psi}$ adds a linear restoring force that pulls the field back toward zero, acting like friction. The subsequent normalisation then ensures the field stays on the unit sphere, making damping effectively a direction-preserving force.

3.5 What Does the Psi Field Produce?

The combination of these mechanisms produces a rich tapestry of phase patterns — think of interference fringes, vortices, and domain walls — which continuously shift and evolve over time. These patterns are the *physical substrate* on which the chemical reactions, energy flows, and life processes are built. In the rendered image (Figure 3), the hue of each cell is directly derived from the local phase φ, which is why the world shows sweeping rainbow-coloured domains with sharp boundaries: those boundaries are the "walls" between regions of different phase, exactly like domain walls in ferromagnets or vortices in superfluids.

4 Thermodynamics: Energy, Heat, and the Drive to Live

4.1 Why Thermodynamics?

Life in the real world is fundamentally a thermodynamic phenomenon. Living organisms maintain themselves far from thermal equilibrium by continuously consuming free energy (sugar, sunlight) and expelling waste heat. Our simulation captures this essential feature: *free energy* F is a usable resource, and *heat* T is a form of degraded, unusable energy.

Free energy vs heat

Think of free energy as the height of water stored behind a dam: it is a source of ordered, directional energy you can exploit to do work. Heat is the water after it has passed through the turbine and spread out below: the energy is still there, but it is now distributed uniformly and cannot drive a turbine.
The key measure of how much "useful" energy is in a system is the *free energy* F, not the total energy $F + T$. Life needs F, not just energy.

4.2 Source Map: Where Does Energy Come From?

The simulation has four fixed "stars" — point-like energy sources defined by Gaussian blobs in the grid:

$$\mathcal{S}(x,y) = \sum_{k=1}^{4} A_k \exp\left(-\frac{(x-c_{x,k})^2 + (y-c_{y,k})^2}{2\sigma_k^2}\right),$$

with amplitudes $A \in \{1.20, 0.92, 0.88, 0.76\}$ and widths $\sigma \in \{0.15, 0.22, 0.27, 0.18\}$. This map is stored as `source_map` and never changes. It plays the role of stellar radiation in a real ecosystem: regions near the "stars" are energy-rich and are the most likely nurseries for life.

4.3 Edge Cooling

The boundary of the grid acts as a heat sink:

$$\Lambda(x,y) = 0.10 + 0.28\, r(x,y), \quad r = \sqrt{x^2+y^2},$$

stored as `edge_cooling`. The outermost cells have $\Lambda \approx 0.38$, losing heat almost four times faster than the centre. This creates a spatial temperature gradient that drives convection-like energy flows from the hot centre toward the cool rim.

4.4 The Thermodynamic Update Equations

The `ThermodynamicsEngine` runs after physics each step:

Free energy and heat evolution

$$F \leftarrow \text{clamp}\Big(F + 0.19\,\nabla^2 F + 0.16\,\mathcal{S} - 0.08\,F - D,\ 0,\ 1.8\Big), \tag{5}$$

$$T \leftarrow \text{clamp}\Big(T + 0.14\,\nabla^2 T + 0.80\,D - \Lambda\,T - 0.14\,R,\ 0,\ 1.8\Big), \tag{6}$$

followed by nonlinear coupling:

$$T \ \mathrel{+}= \ 0.01\ \text{ReLU}(F - 1.2), \tag{7}$$

$$F \ \mathrel{-}= \ 0.02\ \text{ReLU}(T - 1.0), \tag{8}$$

where D is the **energy demand** (set by organisms), R is the **repair drive** (organisms self-maintaining), and $\text{clamp}(x,a,b) = \max(a, \min(b,x))$.

Each term has a clear physical role:

- **Diffusion** (∇^2): energy spreads from high-concentration regions to low ones (Fick's law).
- **Source** ($0.16\,\mathcal{S}$): energy injection near stars.
- **Decay** ($-0.08\,F$): free energy naturally degrades (spontaneous dissipation).
- **Metabolic load** ($-D$): organisms consume free energy; most of it becomes heat ($+0.80\,D$ in the heat equation). The 0.80 factor means 80% of metabolic work ends up as waste heat — a realistic thermodynamic efficiency bound.

- **Cooling** ($-\Lambda T$): edge heat loss.
- **Repair** ($-0.14\,R$): organisms that invest in repair actively cool their local environment (e.g., a biological analogy: sweating).
- **Nonlinear coupling**: if free energy becomes very high ($F > 1.2$), some spills over into heat (energy overflow). If heat becomes very high ($T > 1.0$), it starts destroying free energy (thermal degradation of fuel).

Why these equations produce interesting behaviour

Notice that free energy and heat are not independent: organisms that metabolise vigorously (D large) simultaneously drain F and increase T. High T then inhibits life (as we will see in the next section). This creates a *negative feedback loop* that naturally limits runaway growth — exactly the kind of self-regulation seen in real ecosystems. Meanwhile, spatial diffusion lets energy-rich regions near stars "export" free energy to neighbouring cells, seeding new growth far from the source.

5 The Chemical Life Engine: Gray-Scott Reaction-Diffusion

5.1 A Brief History of Patterns from Nothing

In 1952, Alan Turing — yes, that Turing — published a remarkable paper titled "The Chemical Basis of Morphogenesis." He showed that two chemicals diffusing through tissue and reacting with each other could spontaneously produce *spatial patterns*: stripes, spots, spirals. This was counterintuitive because diffusion alone should *blur* patterns, not create them. The trick is that the two chemicals play opposite roles: one activates itself (positive feedback), the other inhibits the activator (negative feedback), and crucially they diffuse at *different rates*. These are called **Turing patterns** or **reaction-diffusion** patterns.

In 1984, John Gray and Scott Pearson studied a specific reaction scheme (the **Gray-Scott model**) that has become one of the canonical sandboxes for studying how richly varied spatial patterns can emerge from just two coupled PDEs.

5.2 The Gray-Scott Model

Consider two chemical species a ("substrate", abundant) and b ("activator", rare at first). The reaction is:

$$a + 2b \;\rightarrow\; 3b \quad \text{(autocatalytic production of } b\text{)},$$

meaning one molecule of a combines with two of b to make three b's — b catalyses its own production. The rate of this reaction is proportional to $a \cdot b^2$ (one factor of a, two of b).

The feed/kill terms keep the system out of equilibrium: a is continuously replenished from an external reservoir at rate f, and b is removed at rate k.

The Gray-Scott PDEs are:

$$\partial_t a = D_a\,\nabla^2 a - ab^2 + f(1-a), \tag{9}$$

$$\partial_t b = D_b\,\nabla^2 b + ab^2 - (k+f)\,b. \tag{10}$$

Intuition for Gray-Scott

Think of a as grass and b as rabbits. Rabbits eat grass (the $-ab^2$ term), and rabbit populations grow in proportion to how much grass they eat (the $+ab^2$ term). Grass grows back from rain (feed f), and rabbits die at rate k. The crucial twist: grass diffuses slowly ($D_a > D_b$ is not always true, but the asymmetry is key), so rabbit populations can outpace local grass regrowth, creating clumps of rabbits separated by grass-rich plains. Over time this produces spots, stripes, or labyrinthine patterns depending on f and k.

5.3 The Extended Gray-Scott in This Simulation

The simulation's `CellularLifeEngine` extends the basic Gray-Scott model in several important ways:

Coupling to the physical substrate. The feed rate f and kill rate k are not constants but spatially varying fields:

$$f = 0.024 + 0.020\,\mathcal{S} + 0.016\,\kappa, \tag{11}$$

$$k = 0.048 + 0.020\,\hat{T} + 0.010\,(1-\kappa), \tag{12}$$

where $\hat{T} = T/\max(T)$ is normalised heat and κ is the field coherence from the physics engine. Stars feed the chemistry; heat kills it; coherent field regions are more hospitable to life.

A third chemical: nutrient. A third species, nutrient n, is added:

$$\partial_t n = D_n\,\nabla^2 n + 0.12\,\mathcal{S} - 0.08\,n - 0.05\,ab^2. \tag{13}$$

Nutrient is produced near stars and consumed by the reaction. It provides an additional food source that modulates the substrate-push term:

$$\text{substrate push} = 0.05\,\sigma\big(\sin(2.4\varphi) + 1.1\kappa\big)\cdot n, \tag{14}$$

which injects extra activator b in phase-coherent, nutrient-rich regions. This couples the abstract physics field directly to the chemistry — certain phase patterns become "fertile ground" for life.

The full chemical update.

Extended Gray-Scott equations

$$a' = a + 0.22\,\nabla^2 a - ab^2 + f(1-a), \tag{15}$$

$$b' = b + 0.11\,\nabla^2 b + ab^2 - (k+f)b + \text{substrate push}\cdot n, \tag{16}$$

$$n' = n + 0.14\,\nabla^2 n + 0.12\,\mathcal{S} - 0.08\,n - 0.05\,ab^2. \tag{17}$$

Note $D_a = 0.22 > D_b = 0.11$: the substrate diffuses twice as fast as the activator — a classic Turing instability condition.

5.4 Biomass and Membrane

The activator concentration b alone does not constitute "life" in the simulation. A separate **biomass** field β integrates the life-sustaining conditions:

$$\text{life potential} = \sigma\big(4.3(b - 0.22) + 2.4(\hat{F} - 0.30) - 3.1\hat{T} + 1.8\kappa + 0.8\sin(2.4\varphi)\big), \quad (18)$$

where σ is the logistic sigmoid. This equation is a *biological viability score*: high b (lots of activator), high free energy $\hat{F}$, low heat $\hat{T}$, high coherence κ, and a favourable phase φ all increase it. If even one of these is strongly unfavourable, σ drives the score toward zero.

Biomass evolves as:

$$\beta \leftarrow \text{clamp}\Big(\beta + 0.12\,\nabla^2\beta + 0.34\,(\text{life potential} - \beta) + 0.14\,b\ \text{ReLU}(F - 0.18),\ 0,\ 1\Big). \quad (19)$$

The $0.34(\text{life potential} - \beta)$ term drives biomass toward the life-potential target (logistic-like growth). The $0.14\,b\ \text{ReLU}(F - 0.18)$ term provides additional direct biomass growth whenever there is both activator and free energy. The diffusion $\nabla^2\beta$ lets biomass spread spatially.

The **membrane** field μ is then derived from β and b:

$$\mu \leftarrow 0.82\,\mu + 0.18\,\sigma(5.0(\beta - 0.45)) \cdot \sigma(4.0(b - 0.18)). \quad (20)$$

This is a slow exponential moving average (0.82 coefficient $\approx$ time constant of ≈ 6 steps) of whether local conditions are simultaneously rich in both biomass and activator — exactly the conditions that would define a boundary between "inside" and "outside" of a cell. High μ delineates the edges of living structures.

5.5 Signal Field

Finally, organisms communicate through a chemical **signal** field s:

$$s \leftarrow \text{clamp}\Big(s + 0.16\,\nabla^2 s + 0.22\ \text{ReLU}(\beta - 0.34) \cdot \tanh(\psi_2 + 0.3\psi_3) + 0.36\,R_s - 0.20\,s,\ -1,\ 1\Big), \quad (21)$$

where R_s is the `signal_drive` deposited by organism actions. Active biomass regions ($\beta > 0.34$) modulated by the psi field spontaneously emit signal; organisms can additionally boost it. Signal decays ($-0.20\,s$) but also diffuses, so "calls" from one organism can reach neighbours several cells away.

6 Organism Manager: Detecting Life from the Field

6.1 The Life Mask

Every few steps (default: every 4 steps), the `OrganismManager` scans the continuous fields for regions that meet all biological viability criteria simultaneously:

$$\text{mask}(x, y) = [\beta > 0.54]\ \wedge\ [F > 0.16]\ \wedge\ [T < 0.90]\ \wedge\ [b > 0.14]. \quad (22)$$

This is a *threshold-based classifier* with four features. The thresholds are chosen so that only genuinely healthy, energy-rich, cool, and chemically active cells are labelled "alive." A cell in a blazing-hot region, or one with no free energy, or one where the Gray-Scott chemistry has stalled, will fail the mask test.

6.2 Connected-Component Labelling

Given the binary mask, the engine performs **connected-component labelling** using a breadth-first search (BFS):

1. Scan the mask raster-by-raster.
2. On finding an unlabelled live cell, start a BFS queue.
3. Expand to 4-connected neighbours (up, down, left, right) that are also alive and unlabelled.
4. Once the BFS terminates, record the connected blob as one *component* (potential organism).
5. Discard blobs with fewer than 18 pixels (noise filter).

For each surviving component, compute statistics: centroid, bounding box, area, mean signal, mean heat, mean free energy, mean biomass.

Why connected components?

In real biology, a cell (or organism) is defined by its *spatial contiguity* — a connected region of living material separated from other living regions by dead space. BFS is the simplest algorithm that detects this: it finds all cells reachable from a seed without leaving the living region. The resulting components are the simulation's "organisms": not hand-coded agents, but *emergent spatial structures* detected from the field.

6.3 Organism Record Lifecycle

Once components are detected, the manager *matches* them to existing tracked organisms using a simple nearest-centroid assignment (Hungarian-style greedy):

- Compute distance between every component centroid and every live organism centroid.
- Sort candidate pairs by distance; greedily assign closest pairs.
- A match is valid only if distance $\leq 18 + \sqrt{\max(\text{area})}$.
- Matched organisms update their centroid, velocity, area, and energy reserve (via a blended formula):

$$E \leftarrow \max\big(0,\ 0.82\,E + 0.36\,\bar{F} + 0.24\,\bar{\beta}\big).$$

- Unmatched components spawn new organisms with random genomes.
- Organisms not seen for > 16 steps, or with $E < 0.015$, are killed (biomass and signal suppressed locally).

Diversity is measured as the mean pairwise *genome distance* across all alive organisms:

$$\text{diversity} = \frac{1}{\binom{n}{2}} \sum_{i<j} d_G(\text{genome}_i, \text{genome}_j),$$

where d_G averages the absolute differences in 7 scalar genome traits (metabolism, heat tolerance, motility, etc.).

7 Genomes: The Heritable Blueprint

Each organism carries a **genome** — a collection of heritable parameters that determine its physiology and neural wiring.

7.1 Scalar Traits

Trait	Range	Meaning
`metabolism`	$[0.5, 1.6]$	Scales energy harvesting rate
`heat_tolerance`	$[0.05, 1.0]$	Below this heat, dreaming is allowed
`motility`	$[0.05, 1.3]$	Scale of movement per action
`signaling`	$[0.05, 1.5]$	Scale of chemical signal emitted
`cooperation`	$[0.0, 1.2]$	Willingness to share energy
`reproduction_threshold`	$[0.4, 1.4]$	Min energy to reproduce
`mutation_scale`	$[0.02, 0.18]$	Width of perturbation at mutation
`hue_shift`	$[-0.3, 0.3]$	Visual colour offset (no functional role)
`brain_scale`	$[0.5, 1.6]$	Scales neural output magnitude
`dream_gain`	$[0.0, 0.45]$	Learning rate during dream replay

Table 2: Scalar genome traits and their roles.

7.2 Neural Weight Traits

The genome also encodes the *weights and biases* of the organism's neural network:

- $W_1 \in \mathbb{R}^{11\times 12}$, $\mathbf{b}_1 \in \mathbb{R}^{12}$ (input-to-hidden layer),
- $W_2 \in \mathbb{R}^{12\times 6}$, $\mathbf{b}_2 \in \mathbb{R}^{6}$ (hidden-to-action layer),
- World model $W_m \in \mathbb{R}^{17\times 11}$ (predicts next observation from current obs+action).

The neural architecture is thus a 2-layer **multi-layer perceptron (MLP)** of input dimension 11, hidden dimension 12, output dimension 6.

7.3 Initialisation and Mutation

New organisms (born from a previously unoccupied connected component) receive a *random genome*: scalar traits drawn uniformly from their ranges, weight matrices drawn from centred Gaussians.

When an organism reproduces, its offspring get a **mutated genome**:

$$\text{trait}' = \text{clamp}\big(\text{trait} + \mathcal{N}(0, \sigma_m^2),\ \text{lo}, \text{hi}\big), \tag{23}$$

$$W' = W + \mathcal{N}(0, \sigma_m^2 I), \tag{24}$$

where σ_m is `mutation_scale` from the parent genome. The mutation scale *itself* mutates, so organisms can evolve higher or lower evolvability — this is **evolution of the mutation rate**, a feature of real organisms.

Sexual reproduction (when a partner is found via the social network) produces a child via genome blending:

$$G_{\text{child}} = \text{mutate}\left(\frac{G_a + G_b}{2}\right).$$

All scalar traits and all matrix entries are averaged element-wise, then perturbed.

8 The Neural Brain: Perception, Action, and Dreaming

8.1 Observation: What Can an Organism See?

At each step, each organism forms an **observation vector** $\mathbf{o} \in \mathbb{R}^{11}$ by reading the field values at its centroid grid cell:

Index	Value	Meaning
0	$F(\text{centroid})$	Local free energy
1	$T(\text{centroid})$	Local heat
2	$s(\text{centroid})$	Local signal
3	$\beta(\text{centroid})$	Local biomass
4	$a(\text{centroid})$	Substrate concentration
5	$b(\text{centroid})$	Activator concentration
6	$n(\text{centroid})$	Nutrient concentration
7	E	Own energy reserve
8	$\min(\text{age}/200, 1)$	Normalised age
9	$\min(\deg/8, 1)$	Social connectivity (number of edges)
10	$\mathcal{U}(-1, 1)$	Random noise

Table 3: The 11-dimensional observation vector.

The random noise dimension (10) prevents the network from collapsing to a deterministic mapping when all other inputs are identical — it is a source of behavioural diversity.

8.2 Action: The Neural Forward Pass

The neural network computes actions as a two-layer tanh MLP:

$$\mathbf{h} = \tanh\big(\mathbf{o}\, W_1 + \mathbf{b}_1\big) \in \mathbb{R}^{12}, \tag{25}$$

$$\mathbf{a} = \text{brain_scale} \cdot \tanh\big(\mathbf{h}\, W_2 + \mathbf{b}_2\big) \in \mathbb{R}^{6}. \tag{26}$$

Why tanh activation?

The tanh (hyperbolic tangent) function maps any real number to the range $(-1, 1)$. This is useful for two reasons:

1. *Bounded outputs*: no action can be infinitely large, preventing the organism from performing catastrophically destructive moves.
2. *Smooth gradients*: unlike a step function or ReLU, tanh is differentiable everywhere and provides a smooth gradient signal — important if we were doing gradient-based learning (here the learning is Hebbian-style, but smooth activations still help the world-model updates remain stable).

The 6-dimensional action vector controls:

Index	Action	Effect
0	Δx	Move left/right (scaled by motility)
1	Δy	Move up/down (scaled by motility)
2	σ_{emit}	Deposit signal at position (bipolar)
3	h (harvest)	Extra metabolic harvesting
4	r (repair)	Deposit repair drive, cool local heat
5	p (reproduce bias)	Adds to fitness, nudging reproduction

Table 4: The 6-dimensional action space.

Applying actions. Movement is implemented by updating the centroid:

$$(x', y') = \text{clamp}\big((x + 2.8\,\text{motility} \cdot a_0,\ y + 2.8\,\text{motility} \cdot a_1),\ 0, N-1\big).$$

Signal, energy demand, repair, and biomass are **deposited as Gaussian patches** onto the corresponding grid fields at the organism's new position:

$$\text{field}(x', y')\ \ +=\ \ A \cdot \exp\Big(-\frac{\|(x, y) - (x', y')\|^2}{2\sigma^2}\Big).$$

This Gaussian spreading is crucial: organisms affect not just their own cell but a small neighbourhood, allowing nearby organisms or cells to sense and react to a moving agent's presence.

Energy bookkeeping. The organism's energy reserve changes as:

$$E_{\text{gain}} = \max\big(0,\ F_{\text{local}} \cdot (0.12 + 0.10\,h) - T_{\text{local}} \cdot 0.06\big), \tag{27}$$

$$E_{\text{cost}} = 0.02 + 0.018(|a_0| + |a_1|) + 0.03|\sigma_{\text{emit}}| + 0.04\,r, \tag{28}$$

$$E \leftarrow \max(0,\ E + E_{\text{gain}} - E_{\text{cost}}). \tag{29}$$

Motion, signalling, and repair all cost energy. Sitting still in a free-energy-rich area is the cheapest strategy — but evolution will favour organisms that spend energy wisely to gain more than they lose.

8.3 Reward and Memory

The organism receives a scalar **reward** signal after each step:

$$r = 0.8\, o_0 - 0.7\, o_1 + 0.4\, o_3 + 0.5\, E. \tag{30}$$

High free energy is good, high heat is bad, high biomass is good, and high energy reserves are good. This reward is stored in the organism's **episodic memory** alongside the observation, action, and next observation:

$$\text{memory} = \{(\mathbf{o}_t, \mathbf{a}_t, r_t, \mathbf{o}_{t+1})\}_{t=0}^{T_{\max}-1},$$

with a maximum of 32 entries (oldest discarded first).

8.4 The World Model: Learning to Predict

Every step, the organism updates its **world model** $W_m \in \mathbb{R}^{17\times 11}$ using the most recent experience tuple $(\mathbf{o}, \mathbf{a}, r, \mathbf{o}')$:

$$\text{input} = \text{clamp}\big([\mathbf{o}; \mathbf{a}],\ -2,\ 2\big) \in \mathbb{R}^{17}, \tag{31}$$

$$\hat{\mathbf{o}}' = \text{clamp}\big(\text{input} \cdot W_m,\ -2,\ 2\big), \tag{32}$$

$$\delta = \text{clamp}\big(\mathbf{o}' - \hat{\mathbf{o}}',\ -1.5,\ 1.5\big), \tag{33}$$

$$W_m \leftarrow \text{clamp}\Big(W_m + \gamma\, \text{input}^T \delta,\ -4,\ 4\Big), \tag{34}$$

where $\gamma = \texttt{dream_gain}$ (a genome trait).

This is a **Hebbian-style update** (or equivalently, one step of online gradient descent on the MSE loss $\|\hat{\mathbf{o}}' - \mathbf{o}'\|^2$): the world model learns to predict the next observation given the current observation-action pair. The update is $\text{input}^T\delta$: an outer product that strengthens the model weights along dimensions where the prediction error was large.

What is a world model for?

A world model is the organism's internal "simulation" of its environment. Instead of always having to physically move somewhere to see what is there, an organism with a good world model can *imagine* the consequences of an action before executing it. This is the foundation of **model-based reinforcement learning**: plan in imagination, execute only the best plan.

8.5 Dreaming: Mental Simulation and Bias Learning

When an organism is in a cool environment ($T < \texttt{heat_tolerance}$) with sufficient energy ($E > 0.25$), it *dreams*: it runs a short planning episode in its world model.

1. Take the most recent observation $\mathbf{o}$ from memory.
2. Generate 4 candidate actions: the current policy output $\mathbf{a}$, plus 3 noisy variants $\mathbf{a} + \mathcal{N}(0, \gamma^2)$.
3. For each candidate $\mathbf{a}_c$, simulate the next state:

$$\hat{\mathbf{o}}' = \text{clamp}\big([\mathbf{o}; \mathbf{a}_c] \cdot W_m,\ -2,\ 2\big).$$

4. Score each candidate using the reward function eq. (30) minus a cost penalty:

$$\text{score}(c) = r(\hat{\mathbf{o}}', E) - 0.03\,\|\mathbf{a}_c\|.$$

5. Pick the best candidate $\mathbf{a}^*$.
6. Nudge the action bias $\mathbf{b}_2$ in the direction of $\mathbf{a}^*$:

$$\mathbf{b}_2 \leftarrow \text{clamp}\big(0.96\,\mathbf{b}_2 + 0.04\,\mathbf{a}^* + 0.01\gamma,\ -2,\ 2\big).$$

This is a **model-predictive control (MPC)** style planning loop with a horizon of 1 step. The bias update is a soft, conservative nudge rather than a full gradient step, preventing the policy from collapsing to a degenerate solution from a single dream. The 0.96 decay keeps old biases from dominating.

The result is that organisms which dream regularly (low heat, high energy, high `dream_gain`) gradually *steer their neural policy* toward actions that their world model predicts to be reward-maximising. This is learning that happens without any interaction with the real environment — purely from internal simulation.

9 The Evolution Engine: Darwin on a Grid

9.1 Reproduction Conditions

Each step, the evolution engine checks every alive organism. An organism can reproduce if:

1. Its age ≥ 10 (prevents immediate self-reproduction).
2. Its energy reserve $E \geq$ `reproduction_threshold` (a genome trait, range $[0.4, 1.4]$).
3. The total alive population is below the cap (32 organisms).

9.2 Partner Selection and Genome Blending

If the organism has social connections (see Section 10), it selects the partner with the highest combined weight + trust − conflict score from its social edges. The partner must also have $E \geq 0.25$.

If a partner is found, **sexual reproduction** occurs:

$$G_{\text{child}} = \text{mutate}\Big(\tfrac{G_{\text{parent}} + G_{\text{partner}}}{2}\Big).$$

If no partner is available, **asexual reproduction** occurs:

$$G_{\text{child}} = \text{mutate}(G_{\text{parent}}).$$

9.3 Child Placement and Field Seeding

The child is placed at a random angle and distance from the parent:

$$\theta \sim \mathcal{U}(0, 2\pi), \quad d = 8 + 10 \cdot \texttt{motility}_{\text{child}},$$

so fast organisms (high motility genome) tend to place children farther away, reducing competition. The birth site is then *seeded with biological deposits*: biomass, activator b,

free energy, and signal are all boosted with small Gaussian patches around the child's position. This gives the newborn a head start.

The parent pays a cost: $E_{\text{parent}} \leftarrow 0.52\, E_{\text{parent}}$ (loses 48% of its energy reserve to the child).

9.4 Death

Organisms die when:

- They are not seen (no matching cluster) for > 16 steps (they have moved off a living region or the region died).
- Their energy reserve falls below 0.015.
- Age > 180 with $E < 0.10$ (senescence).

On death, local biomass is reduced by 30% and local signal by 50% — the biological material decomposes and is partially recycled into the environment.

9.5 What Does Evolution Select For?

The fitness criterion is implicit in the physics:

- Organisms with high `metabolism` extract energy faster but also generate more heat (via the thermodynamic engine), risking overheating themselves.
- High `motility` lets organisms find better spots but costs more energy per step.
- High `signaling` builds social trust faster but costs energy.
- A `reproduction_threshold` that is too low means reproducing on meagre reserves (children are born weak); too high means missing opportunities.

No fitness function is explicitly programmed. Instead, the genome traits interact with the physical environment, and only organisms that "happen" to balance these trade-offs survive long enough to reproduce. Over generations, the population self-organises toward strategies that are locally optimal in the current environment — which itself changes as the population changes. This is **co-evolution**: organisms and environment shape each other.

10 Social Networks: Trust, Cooperation, and Competition

10.1 Building the Social Graph

Every step, the `SocialNetworkEngine` scans all pairs of alive organisms. If two organisms are within distance 34 (in grid cells), they *interact*, and a `SocialEdge` is created or updated between them.

Each edge carries three quantities: **weight** (overall strength of relationship), **trust** (cooperative history), and **conflict** (competitive history). All three decay each step ($\times 0.985$ or $\times 0.992$) unless refreshed, so relationships fade without continued contact.

10.2 Cooperation and Competition

For each interacting pair (i, j):

$$\text{cooperation} = \max(0,\ 1 - |\texttt{signaling}_i - \texttt{signaling}_j|) \cdot 0.03, \tag{35}$$

$$\text{competition} = \max(0,\ 0.03 - 0.0008\, d). \tag{36}$$

Organisms with similar signalling chemistry cooperate more — like-minded signallers recognise each other. Competition is distance-dependent: very close organisms compete for resources; distant ones can coexist.

Energy transfer. If organism i has significantly more energy than j ($E_i > E_j + 0.20$) *and* i has high cooperation (`cooperation` > 0.55), then i transfers up to 3% of its energy reserve to j. This is altruistic sharing — evolutionarily unstable unless it helps kin or cooperating partners survive, and indeed it is only triggered when the social bond (`cooperation` trait) is strong enough.

Edge update.

$$\text{weight} \leftarrow \min(1.5,\ \text{weight} + \text{coop} - 0.5\,\text{comp} + 0.02), \tag{37}$$

$$\text{trust} \leftarrow \min(1,\ \text{trust} + \text{coop}), \tag{38}$$

$$\text{conflict} \leftarrow \min(1,\ \text{conflict} + \text{comp}). \tag{39}$$

10.3 Knowledge Transfer

When trust is high enough (`trust` > 0.35 for basic transfer, > 0.45 for reverse transfer), organisms share *discoveries*: the most recent discovery of a knowledgeable organism is copied into a naive partner's discovery list. This is a model of cultural transmission — learned knowledge propagates through the social network. Discoveries arise from the science engine (Section 11).

11 The Science Engine: In-Universe Symbolic Regression

11.1 The Idea: Organisms That Do Physics

The most whimsical layer of the simulation is the **science engine**. It imagines that the universe's most capable organisms act as physicists, gathering data about their environment and trying to discover the *laws* governing heat and energy.

This is implemented as **symbolic regression via an evolutionary algorithm**: the engine searches for sparse linear equations that best predict the next step's heat and free-energy values from the current step's field measurements.

11.2 Data Collection

Every step, the engine samples 48 random grid locations and records a **feature vector** of 7 quantities:

1. Current field value (heat or free energy)
2. Discrete Laplacian of the field

3. Source map value at that location
4. Biomass
5. Signal
6. Activator concentration b
7. A constant bias term (always 1.0)

The *target* is the corresponding field value at the *next* step. Up to 640 such samples are retained (oldest discarded).

11.3 Evolutionary Candidate Law Search

Every 24 steps (once enough data has accumulated), the engine runs a micro-evolutionary algorithm to find a sparse linear predictor.

A **candidate law** is a triple $(m, \mathbf{c}, b_0)$: a binary mask $m \in \{0,1\}^7$ selecting which features to use, a coefficient vector $\mathbf{c} \in \mathbb{R}^7$, and a bias b_0. The prediction for a sample is:

$$\hat{y} = (m \odot \mathbf{x}) \cdot \mathbf{c} + b_0,$$

where $\odot$ is elementwise multiplication. The loss of a candidate on the dataset is:

$$\mathcal{L}(m, \mathbf{c}, b_0) = \underbrace{\frac{1}{N}\sum_i (\hat{y}_i - y_i)^2}_{\text{MSE}} + \underbrace{0.012 \sum_j m_j}_{\text{sparsity penalty}}.$$

The sparsity penalty (L0 regularisation on the mask) encourages the engine to find *simple* laws (few terms) rather than overfitting with many weak features.

The evolutionary search runs for 14 generations over a population of 24 candidates:

1. Evaluate all candidates, sort by loss.
2. Keep the 8 best (elitism).
3. Fill the population back to 24 by *mutating* survivors: randomly flip one mask bit, perturb coefficients with $\mathcal{N}(0, 0.12^2)$, perturb bias with $\mathcal{N}(0, 0.08^2)$.

11.4 Rendering Laws as Text

The best candidate law is converted to a human-readable string:

```
heat_next = +0.01 +0.00*center +0.98*laplace ...
```

This string is stored in `science_report` and displayed in the science-engine panel of the dashboard. It is also appended to the *discoveries* list of the "lead scientist" — the alive organism with the highest fitness score.

Significance of the science engine

The science engine is a toy model of empirical law discovery: it gathers data, hypothesises parametric equations, tests them against data, and iterates. Crucially, the laws it discovers are *approximately correct* descriptions of how the simulation's own thermodynamic fields evolve — they capture the dominant terms in equations (5) and (6). The "score" reported on the dashboard is $\max(0, 1 - 0.5(\text{heat MSE} + \text{energy MSE}))$, approaching 1 when the discovered laws are accurate. This is the simulation's proxy for scientific progress.

12 Visualisation: Painting the Universe

12.1 The Rendering Pipeline

At each step, the world is rendered as a 192×192 RGB image using an HSV (Hue-Saturation-Value) colour model, then blended with a temporal trail:

$$\text{trail} \leftarrow 0.86\,\text{trail} + 0.14\,\text{RGB}$$

(exponential moving average with time constant ≈ 7 steps). This smooths out per-step flicker and creates "afterglow" on moving structures.

Hue encodes the phase of the psi field:

$$\text{hue} = \left(\frac{\varphi}{2\pi} + 0.5 + 0.11\,b + 0.04\,s + 0.08\,n + 0.03\,m\right) \bmod 1,$$

producing a rainbow sweep across phase domains, with small colour shifts from chemistry and signals.

Saturation increases with chemistry and biology:

$$\text{sat} = \text{clamp}\big(0.46 + 0.54\,\sigma(3b + 2.2\beta + 1.8\mu),\ 0,\ 1\big).$$

Chemically active, biologically rich cells appear vivid; dead, chemistry-free cells appear washed-out.

Value (brightness) integrates free energy, coherence, and biological richness:

$$\text{val} = \text{clamp}\big(0.06 + 0.46\sqrt{\hat{F}} + 0.23\kappa + 0.20\beta + 0.14n + 0.10\mu,\ 0,\ 1\big).$$

Energy-rich, coherent, biologically active cells are bright; dark regions are energy-poor.

Warm/cool overlays. A warm (red-orange) overlay based on heat and source map, and a cool (green-blue) overlay based on nutrients and signals, are blended with the base RGB. This gives an immediate visual read of the energy landscape: heat glows red-orange like embers, signal-rich communication zones glow blue-green.

Bloom and vignette. A bloom effect (Gaussian blur of pixels above 0.56 brightness) adds a soft glow to the brightest structures. A circular vignette darkens the outer ring, drawing the eye toward the centre where the star sources are concentrated.

12.2 Dashboard Layout

The dashboard (see Figure 4) contains:

- **World panel** (left, 58% of width): the rendered universe with organism outlines and ID labels.
- **State panel** (top right): key metrics including step, entropy, free energy, heat, biomass, coherence, population, links, diversity, and science score. Also shows the top 5 organisms by fitness.

- **Signals panel** (centre right): sparkline time-series of entropy, free energy, heat, population, and science score.
- **Social Graph panel**: force-directed-like layout of up to 10 organisms, with edge thickness proportional to relationship weight and colour proportional to trust.
- **Science Engine panel**: live display of the discovered heat law and energy law, the science score, and recent organism discoveries.

13 Reading and Analysing the Simulation Images

13.1 The Toy Universe Showcase (Figure 3)

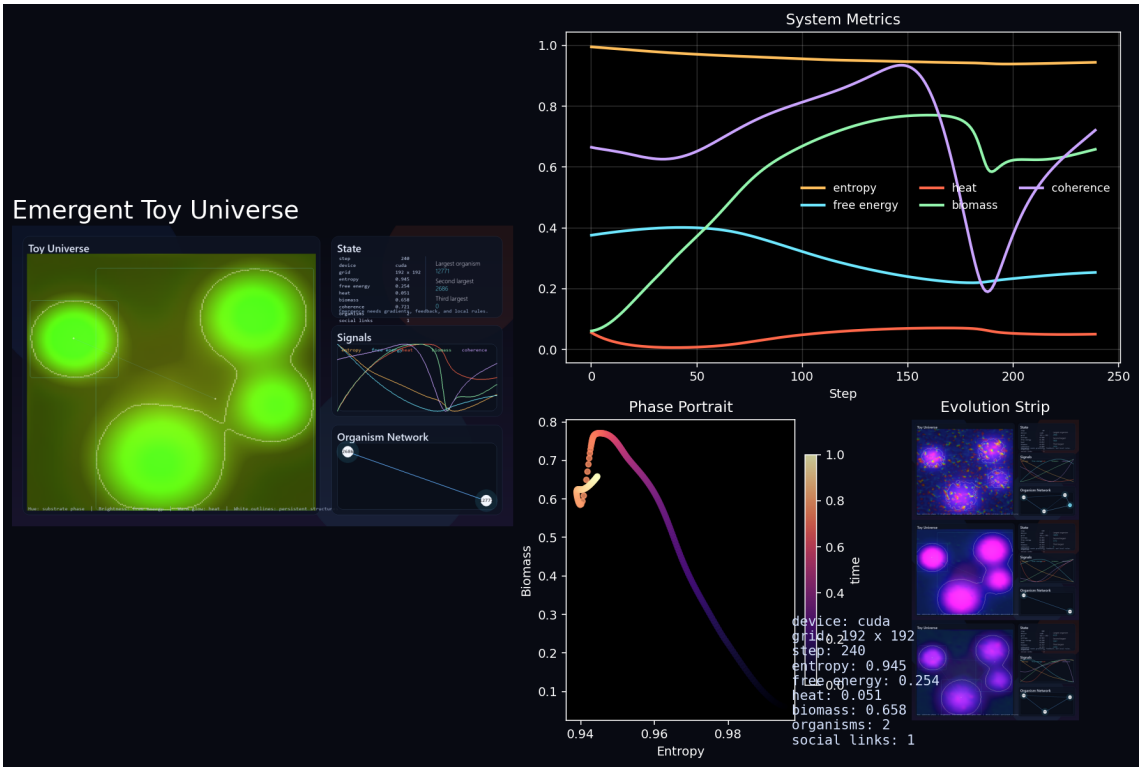

Figure 3: Poster from the simpler `toy_universe.py` simulation (same physics and chemistry, no neural brains or science engine). **Top-right**: System metrics over 250 steps. **Bottom-left**: Phase portrait (entropy vs biomass). **Bottom-right**: Evolution strip (three snapshots).

World panel (large left image). We see four bright, roughly circular "organisms" glowing vivid green-yellow against a dark background. These are the connected components — life-mask blobs — detected at step 250. Their colours (hue from phase, brightness from free energy) are relatively uniform *within* each blob but differ *between* blobs, reflecting that each organism sits in a slightly different phase domain of the psi field. The glowing halos around each blob are the bloom effect from high-brightness pixels. The dark border is the vignette.

White outlines delineate the life-mask boundaries precisely. Lines connecting organism centroids are the social-graph edges: the blue-green colour indicates cooperative relationships (high signal match, low heat).

System metrics (top right). The **entropy** (yellow/orange) curve starts high (random initial state, broadly distributed energy), then decreases as energy concentrates near the star sources, then stabilises. This is classic self-organisation: a disordered initial state spontaneously develops spatial structure, reducing entropy.

The **free energy** (cyan) rises quickly as the source map injects energy, then plateaus once metabolic consumption by the growing biomass balances the source input.

The **heat** (red) grows slowly throughout — the system warms up as metabolism converts free energy to waste heat. That it remains below 0.3 throughout is the signature of effective edge cooling.

The **biomass** (green) follows an S-shaped growth curve: slow initial nucleation, rapid growth, then saturation as the life potential is limited by heat and available free energy.

The **coherence** (purple) starts from random initialisation near 0.5 (equal probability of all four psi channels having moderate norm) and evolves as the orthogonal gates create organised phase domains.

Phase portrait (entropy vs biomass). The scatter plot traces the simulation's trajectory through (entropy, biomass) space. Early frames (dark points) cluster at high entropy, low biomass — random chaos before life takes hold. Later frames (bright points) shift toward lower entropy, higher biomass — the attractor of the system. The arrow-like shape of the trajectory shows that as biomass grows, entropy falls; this is the information-theoretic signature of **biological organisation reducing disorder locally**. (Note: globally the entropy still increases — the heat dissipated through the edges more than compensates. The second law is respected.)

Evolution strip. Three snapshots at steps 90, 180, and 270. The organisms are clearly visible from the first snapshot onwards, confirming that persistent structures form early and are maintained throughout the run. The increasing brightness over time reflects the growing free-energy concentration.

13.2 The Full Emergent Universe Engine (Figure 4)

World panel. The universe in the full simulation looks similar to the toy version in large-scale structure (bright blobs on dark background, phase-coloured domains), but the dynamics are richer: organisms have moved (note the slightly irregular blob shapes from neural-directed motion), membranes are visible as sharper blob boundaries, and the trail effect creates a slight smear showing recent organism trajectories.

The ID labels (numbers in white) identify tracked organisms by their `organism_id`. The fact that IDs are small (1-digit or 2-digit) and correspond to a population of 12 alive organisms (as shown in State panel) means most of the early random organisms have died and been replaced by the fitter evolved descendants.

Population curve (green, system metrics). The most striking feature of the full simulation metrics is the **population oscillation**: it shows large swings (from near 0 to ∼32 and back). These oscillations are a signature of *boom-bust cycles*:

1. Population grows, consuming free energy.

Figure 4: Poster from the full `main.py` simulation at step 240 (CUDA, seed 7, 192×192, science score 0.992). The world panel shows the live dashboard with neural organisms, social graph, and science engine. **Top-right**: System metrics. **Bottom-right**: Discovered laws.

2. Free energy drops; heat rises.
3. Organisms start failing the life mask (heat too high, energy too low).
4. Mass die-off; environment recovers.
5. Free energy rises again; new organisms nucleate.
6. Repeat.

This is a classic predator-prey type oscillation, here between the organisms (consumers) and the energy field (resource), analogous to the Lotka-Volterra equations of ecology.

Science score (purple). The science score climbs steeply and reaches 0.992 — near-perfect law discovery. This means the evolutionary regression algorithm has found a linear combination of the 7 features that predicts the next-step heat and free energy with very high accuracy. Looking at the text panel:

```
heat law:   heat_next = +0.01 +0.00
energy law:   free_next = +0.06 +0.99*center
```

The energy law $F_{t+1} \approx 0.06 + 0.99\, F_t$ captures the dominant effect: free energy at a cell is strongly autocorrelated (yesterday's value predicts tomorrow's, with a small upward drift from the source). This is a quantitatively correct first-order description of the diffusion-and-source dynamics of equation (5). The science engine has "discovered" a conservation-like law governing the universe's energy field.

Social Graph panel. The social graph shows a small network of 6–8 nodes with several edges. Node size is proportional to organism area (larger blobs = larger nodes). Edge colour ranges from blue (low trust) to teal/cyan (high trust). The presence of thick, bright edges means some organisms have maintained sustained cooperative contact over many steps — they have accumulated trust through repeated energy-sharing and signal-compatible interactions. These are the organisms most likely to have undergone sexual reproduction with each other, blending their genomes and sharing discoveries.

14 The Toy Universe vs the Full Engine: A Comparison

Feature	**toy_universe.py**	**main.py (full)**
Physics engine	Simple rotation matrix	Brick-wall orthogonal gates
Organism tracking	Shape + area only	Full OrganismRecord with genome
Neural brains	None	2-layer MLP + world model + dreaming
Evolution	None	Mutation, sexual recombination
Social network	Proximity + signal edges	Full trust/conflict/cooperation
Science engine	None	Symbolic regression via ES
Diversity metric	None	Mean pairwise genome distance
Code lines (approx.)	∼900 (monolithic)	∼1500 (modular)
Emergent phenomena	Phase patterns, GS life	All of the above + ecology

Table 5: Comparison between the two simulation versions.

The toy universe is a clean, self-contained demonstration of how phase patterns, reaction-diffusion chemistry, and free-energy thermodynamics alone produce persistent spatial structures without any higher-level machinery. The full engine builds an entire biological stack on top.

15 Key Mathematical Concepts — Quick Reference

This section collects the essential mathematical ideas used throughout, in one place.

15.1 The Discrete Laplacian and Diffusion

On a 2D grid, the Laplacian operator is discretised as:

$$(\nabla^2 f)_{i,j} = f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - 4\, f_{i,j}.$$

This is implemented as a 3×3 convolution with kernel $\left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{smallmatrix}\right]$. Circular (periodic) boundary conditions wrap the grid edges. Adding $\alpha\, \nabla^2 f$ to f per step implements **discrete diffusion** with diffusivity α. In the continuum limit, this is the heat equation $\partial_t f = \alpha\, \nabla^2 f$.

15.2 The Sigmoid and Its Relatives

The **logistic sigmoid**:

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma : \mathbb{R} \to (0,1).$$

Used for probabilities and smooth thresholds.

The **tanh**:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1, \quad \tanh : \mathbb{R} \to (-1,1).$$

Used for neural activations (bipolar, bounded).

The **ReLU**:

$$\mathrm{ReLU}(x) = \max(0, x),$$

used for one-sided effects ("only if positive").

15.3 Shannon Entropy

Given a probability distribution $p_i = f_i / \sum_j f_j$, the Shannon entropy is:

$$H = -\sum_i p_i \log p_i.$$

Normalised by $\log N$ (where N is the number of cells) to give $H \in [0,1]$. $H \approx 1$ means energy is uniformly spread; $H \approx 0$ means all energy is concentrated in one cell. Biological self-organisation corresponds to *decreasing* entropy as energy concentrates in living structures.

15.4 Orthogonal Matrices from QR Decomposition

To generate a random orthogonal matrix Q:

1. Sample $A \sim \mathcal{N}(0, I_{n\times n})$.
2. Compute QR decomposition: $A = QR$.
3. If $\det(Q) < 0$, flip the sign of one column: $Q_{:,0} \leftarrow -Q_{:,0}$.

This samples uniformly from the Haar measure on $O(n)$ (the uniform distribution on the orthogonal group) — no direction is privileged.

15.5 Gaussian Deposition

Depositing a Gaussian "blob" at position (x_0, y_0):

$$\mathrm{field}(x,y) \;\; += \;\; A \exp\Big(-\frac{(x-x_0)^2 + (y-y_0)^2}{2\sigma^2}\Big).$$

This is the standard Gaussian kernel. In the code, organism actions are deposited as Gaussian blobs so that their effects spread smoothly over a few neighbouring cells.

15.6 Connected-Component Labelling via BFS

Breadth-first search on the binary mask: for each unvisited live cell, expand to 4-connected neighbours iteratively until no more live neighbours remain. Time complexity: $O(N^2)$ per scan (each cell visited once). In practice, for sparse masks on a 192×192 grid, this is very fast.

16 Physical Analogies and Inspirations

The simulation draws on a rich tradition of physics and complex systems:

Quantum field theory The $\boldsymbol{\psi}$ field is inspired by the wave function, with orthogonal gates mimicking unitary time evolution. The phase φ and coherence κ mirror phase and amplitude in quantum mechanics.

Statistical mechanics The thermodynamic engine captures the first and second laws: energy is conserved (flow is tracked), entropy increases globally (waste heat accumulates and escapes). The simulation is a non-equilibrium statistical system maintained by the source map.

Morphogenesis The Gray-Scott chemical system is a direct descendent of Turing's 1952 reaction-diffusion model. The patterns it produces (spots, stripes, spirals) appear in animal coats, sea-shell markings, and embryonic tissue.

Ecology The population oscillations, predator-prey dynamics, cooperation/competition trade-offs, and genetic diversity all have direct analogues in mathematical ecology (Lotka-Volterra, evolutionary game theory).

Cognitive science The MLP neural brain, world model, and dreaming are simplified versions of Hebb's rule, model-based RL, and sleep consolidation in neuroscience.

Cultural evolution The discovery propagation through the social network mirrors the spread of knowledge through human societies.

Philosophy of science The science engine is a computational toy model of Karl Popper's falsificationism and Thomas Kuhn's paradigm shifts, operating inside a simulated universe.

17 End-to-End Walk-Through of One Simulation Step

Let us trace exactly what happens in one time step of the full engine, starting from state $\mathbf{S}_t$:

1. **Physics.** The psi field $\boldsymbol{\psi}$ is updated via the orthogonal gate (horizontal or vertical, alternating), then diffusion, mixing, curl, drive, and damping (equation (4)). Phase φ, coherence κ, and matter type m are recomputed.

2. **Thermodynamics.** Snapshots of F_t and T_t are saved. New F_{t+1} and T_{t+1} are computed (equations (5)–(6)) using energy demand D_t set by organisms in the previous step.

3. **Cellular Life.** Chemical species a, b, n are evolved by the extended Gray-Scott equations. Biomass β and membrane μ are updated. Signal s is updated; energy demand D is augmented by $0.02\,ab^2 + 0.03\ \mathrm{ReLU}(\beta - 0.30)$.
4. **Organism Manager** (every 4 steps). The life mask is computed. BFS finds connected components. Existing organisms are matched or aged. Unmatched clusters spawn new organisms. Stale/starved organisms die. Diversity is recomputed.
5. **Drive reset.** Energy demand, signal drive, and repair drive are zeroed so organisms start fresh this step.
6. **Neural Brain.** Each alive organism:
   (a) Observes its local field values.
   (b) If memory is non-empty: compute reward, store experience, update world model (equation (34)).
   (c) Compute action via MLP (equation (26)).
   (d) Apply action: move, deposit signal/repair/energy demand.
   (e) Update energy reserve.
   (f) If cool and energy-rich: dream (MPC planning).
7. **Evolution.** Each age-eligible organism with sufficient energy reproduces. Partner selection, genome blending, or mutation. Child organism placed and seeded. Senescent starved organisms killed.
8. **Social Network.** All edges decay. Nearby pairs interact: cooperation, competition, energy transfer. Knowledge is transferred along high-trust edges.
9. **Science Engine.** 48 new field samples are appended to the dataset. Every 24 steps, evolutionary regression finds the best linear law for heat and energy. Law is stored; lead scientist's discoveries updated.
10. **Metrics.** Entropy, free energy, heat, biomass, coherence, membrane, population, links, diversity, and science score are appended to history.
11. **Rendering.** The HSV composite image is computed from phase, chemistry, and energy fields. The trail is updated. The dashboard frame is rendered. The frame is stored for GIF output.

18 Conclusion: What Has the Simulation Achieved?

The Emergent Universe Engine is a tour de force of layered complexity. Starting from nothing but a random Gaussian field and a fixed star map, it spontaneously develops:

- **Physical structure:** phase domains, vortices, and domain walls in the psi field, driven by the orthogonal gate dynamics.
- **Chemical patterns:** Turing-like spots and stripes in the activator field b, mediated by the asymmetric diffusion of the Gray-Scott system.
- **Energetic gradients:** steady-state spatial distributions of free energy and heat, maintained far from equilibrium by the four star sources and edge cooling.

- **Living structures:** persistent, self-maintaining regions of high biomass and membrane formation, identified as organisms via connected-component labelling.
- **Behaviour:** organisms move, harvest, signal, and repair, guided by neural networks whose weights are part of their heritable genome.
- **Learning:** world models are updated online from experience; dreaming refines action biases toward reward-maximising policies — all without any pre-programmed objective.
- **Darwinian evolution:** the population's genome distribution drifts under selection pressure imposed by the thermodynamic environment, with both asexual mutation and sexual recombination as generators of variation.
- **Social complexity:** organisms form trust relationships, cooperate, compete, and share knowledge, producing an emergent social network.
- **Science:** the simulation's own organisms effectively rediscover approximate quantitative laws describing how its fields evolve — an in-universe empirical science program.

None of these phenomena are explicitly programmed. Each is an *emergent consequence* of the layer below it. Phase patterns enable chemical patterns; chemical patterns enable life; life enables evolution; evolution enables intelligence; intelligence enables social networks; social networks enable cultural knowledge diffusion.

The central lesson

Emergence is not a mystery — it is a consequence. Given the right substrate (energy gradients, nonlinear interactions, diffusion, feedback), complexity does not have to be designed: it *has* to appear. The question for the designer of a complex system is not "how do I make this happen?" but "how do I not accidentally prevent it from happening?" The Emergent Universe Engine answers that question by getting out of the way: it provides minimal ingredients and lets the physics do the rest.

The simulation's code is a remarkably concise implementation of this philosophy. Each engine module is ∼50–150 lines of PyTorch. The physics fits in one function. The thermodynamics in ten lines. The cellular life in ∼25 equations. Yet together they produce a system whose long-term behaviour nobody fully predicted when writing the equations — which is, of course, exactly the point.

Simulation parameters used in the figures: grid size 192×192; 360 simulation steps; random seed 7; device: CUDA (NVIDIA GPU); internal dimension $d = 4$; chemical dimension 3; hidden layer size 12; action size 6; observation size 11; max organisms 32; max memory 32; sample every 4 steps; organism refresh every 4 steps; science period 24 steps; FPS 30.